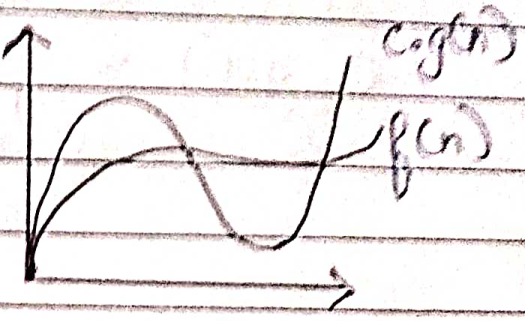
Unit 1 :-

Mathematical foundations & Asymptotic Notation

1] Big O notation (upperbound)

→ Worst case.

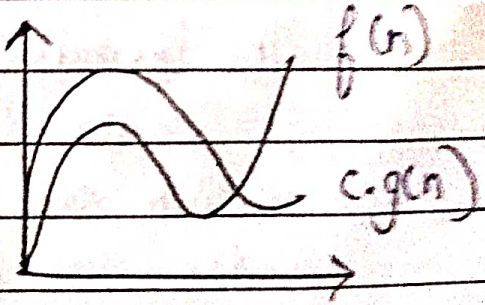


$f(n) < c \cdot g(n) \quad c > 1, n > 0.$

$$\boxed{f(n) = O\ g(n)}$$

2] Big Ω notation (lowerbound)

→ best case.

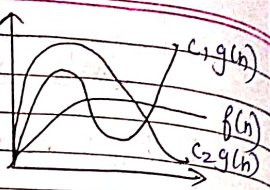


$f(n) > c \cdot g(n), \quad c > 1, n > 0.$

$$\boxed{f(n) = \Omega\ g(n)}$$

3] Big Θ notation & (tight bound)

$f(n) = \Theta g(n)$



$c_2 g(n) < f(n) < c_1 g(n)$

$c_1, c_2 > 1 \quad , n > 0$

① $f(n) = 2n+3$.

$f(n) = O(n)$

$f(n) = \Omega(n)$

$f(n) = \Theta(n)$

★ Characteristics of Algorithm

The formal defination of an algorithm is that it contain the finite set of instruction which are being carried in a specific order to perform specific task. It is not the complete program code. It is the soln (logic) of the problem which can be represented either as an informal description using a flowchart or pseudocode.

① Clear and unambiguous (confusion)
② Well-defined inputs
③ Well-defined output
④ Finiteness
⑤ Flexible
⑥ Language independent

Q. Find upperbound. constant function

① $f(n) = 6599$

→ To find upperbound of $f(n)$ we have to find c and n_0 such that

$0 \leq f(n) \leq c.g(n) \quad , c > 0 \quad n \geq n_0$

$f(n) = 1 \times (8000)$

$6599 < 8000$

$f(n) = O(g(n))$

$= O(8000)$

② $f(n) = 2n+3$

→ To find upperbound of $f(n)$ we have to find c and n_0.

$\Leftarrow$

$2n+3 \leq 3n$

n	f(n)	c.g(n)	
1	5	3	F
2	7	6	F
3	9	9	T
4	11	12	T
5	13	15	T

$\therefore n \geq 3$

$c = 3$ | $n_0 \geq 3$ | $f(n) = O(n)$

③ $f(n) = 6n+3$ (find upperbound)

→ To find the upperbound of $6n+3$ we have to find c & n

$0 \leq f(n) \leq c \cdot g(n) \quad \because c > 0 \quad n \geq n_0$ Imp

$6n+3 \leq 7n$

n	f(n)	g(n)	
1	9	7	F
2	15	14	F
3	21	21	T
4	27	28	T

∴ We got $c=7$ and $n \geq 3$

$f(n) = O(g(n))$

$\boxed{f(n) = O(n)}$

④ $f(n) = 3n^2+2n+4$

→ To find the upperbound of $3n^2+2n+4$ we have to find c & n

$0 \leq f(n) \leq c \cdot g(n)$

$3n^2+2n+4 \leq 4n^2$

	n	f(n)	g(n)	
✗	1	9	4	F
	2	20	16	F
	3	37	36	T
	4	60	64	T

∴ We got $c=4$ and $n \geq 3$.

$f(n) = O(g(n))$

$\boxed{f(n) = O(n)}$

Bigoh notation → $0 \leq f(n) \leq c \cdot g(n)$

$\boxed{f(n) = O(g(n))}$ where $n \geq n_0$ $c>0$ $n_0 \geq 1$

Big (Θ) → $c_1 g(n) \leq f(n) \leq c_2 g(n)$

$\boxed{f(n) = \Theta(g(n))}$ where $n \geq n_0$, $c_1 c_2 > 0$ $n_0 \geq 1$

Bfg (Ω) → $0 \geq f(n) \geq c \cdot g(n)$

where $n \geq n_0$, $c>0$ $n_0 \geq 1$.

$\boxed{f(n) = \Omega(g(n))}$

① Find lowerbound, $f(n) = 2n+3$

→ To find the lowerbound of $2n+3$ we have to find c & n

$0 \geq f(n) \geq c \cdot g(n)$

$2n+3 \geq 1n$

n	2n+3	n	
1	5	1	T
2	7	2	T

$\boxed{c=1}$ $\boxed{n \geq 1}$

$\boxed{f(n) = \Omega(n)}$

for Θ :-

$n \leq 2n+3 \leq 3n$

$f(n) = \Theta(n)$

where $c=3$;

$n_0 \geq 3$.

★ Divide & Conquer Strategy (DA and C)

1. DA and C(P)
2. {
3. if n is small, solve it
4. else
   {
   divide p into sub problem
   i.e $p_1, p_2 \cdots p_k$
5. Apply DA and C(P) to each of these sub problem
6. Combine all sol^n
7. }
8. }

★ $T(n) = 2T(n/2) + n$ where $n/2$ →

$T(n) = 2[T(n/2) + n$ → (1)

$T(n/2) = 2T(n/2_2) + n/2$

$T(n/2) = 2T(n/4) + n/2$

↳ Put in eq^n (1)

$T(n) = 2[2T(n/4) + n/2] + n$

$= 4T(n/4) + 2(n/2) + n$

$T(n) = 2^2 T(\frac{n}{2^2}) + 2n$

$= 3^2 T(\frac{n}{3^2}) + 3n$

$= 4T(n/4) + 2n$

$= 8T(n/8) + 3n$

$= nT(\frac{n}{n}) + n$

$= k \cdot n \longrightarrow n = 2^k$

$= \log n \cdot n \qquad k = \log_2 n$

$\boxed{T(n) = n \log n}$

★ Binary Search

Index →	0	1	2	3	4	5	6
	10	20	30	40	50	60	70

2] LB, UB, mid, n

3] while (low ≤ high)

LB = 0 UB = n-1 ⇒ 6

$mid = \frac{LB + UB}{2}$

$mid = \frac{6}{2} = 3$

LB	UB
0	6

4] If key = 20
if (a[mid] == key)
a[3]
40 == 20 → Not satisfied
Not find

5] else if (a[mid] > key)
a[3] > 20
40 > 20 → Right condn
high = mid - 1
LB = 0 UB = 3

6] mid = (LB + UB)/2 = (0+2)/2 = 2/2 = 1
a[mid] == key
a[1] == 20
20 == 20
↳ Right condn.
∴ Element found

★ key == 80

3] LB = 0 , UB = n - 1 ⇒ 6
mid = (LB + UB)/2 = 6/2 = 3

4] if (key == 80)
if (a[mid] == key)
a[3] == 80
40 == 80 → Not satisfied

5] else if (a[mid] > key) a[3] > 80 40 > 80
↳ Not satisfied

6] else (a[mid] < key)
a[3] < 80 → 40 < 80
low = mid + 1
LB = 4 UB = 6
mid = (4+6)/2 = 10/2 = 5

7] if (a[mid] == key)
a[5] == 80
80 == 80 → Not satisfied

8] else if (a[mid] > key)
a[5] > 80
60 > 80 → Not satisfied.

9] else
(a[mid] < key)
a[5] < key
60 < 80 → satisfied.
LB = mid + 1 → LB = 6

10] mid = (6+6)/2 = 12/2 = 6.

11] a[mid] == key → Not satisfied.
12] a[mid] > key → Not satisfied
13] a[mid] < key → Not satisfied.
∴ Element not found ∴ Worst case

Page No.
Date : / /

① $T(n) = 2T(n/2) + n$. , $n=1$ $n>1$

→ By using substitution method

Assume

$T(n) = n \log_2 n$

Forward method →

Put $n = n/2$ in eqn ①

$T(n/2) = 2T\left(\frac{n/2}{2}\right) + n/2 \longrightarrow ②$

Put eqn ② in eqn ①

$T(n/2) = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n$

$= 4T\left(\frac{n}{4}\right) + 2\left(\frac{n}{2}\right) + n$

$= 4T(n/4) + 2n$

$= aT(n/a) + 2n.$

$T(n) = n \log_2 n.$

$T[n/2] = [n/2 \log_2 n/2]$

Backward method

By using substitution method

Assume

$T(n) = n \log_2 n.$

$T(n) = 2T(n/2) + n.$

Page No.
Date : / /

$T(n/2) = 2\left[\frac{n}{2} \log_2 \frac{n}{2}\right] + n$

$= 2n \log_2 \left(\frac{n}{2}\right) + n$

$= n\left[\log_2 n - \log_2 2\right] + n$

$= n\left[\log_2 n - 1\right] + n$

$= n \log_2 n - n + n$

$= n \log_2 n.$

Time complexity.

$\boxed{T(n) = O\left(n \log_2 n\right)}$

① $T(n) = 2T(n^2) + 1$

→ $T(n) = 2T(n^2) + 1 \longrightarrow ①$

Assume

$\boxed{T(n) = \log_2 n.}$

Put $n = n^2$

$T(n^2) = \log_2 n^2$

eq. n ① ⇒

$T(n) = 2\left[\log_2 n^2\right] + 1$ $\{\because \log a^b = b \log a\}$

$= 2 \times 2\left[\log_2 n\right] + 1$

$= 4 \log_2 n + 1$

$T(n) = 4 \log_2 n$

$\boxed{T(n) = \log_2 n}$

∴ time complexity $T(n) = O(\log_2 n)$

$T(n) = T(n-1) + n \longrightarrow (1)$

Replace $n = n-1$

$T(n-1) = T(n-1-1) + (n-1)$

$= T(n-2) + (n-1) \longrightarrow (2)$

Putting eqn (2) in eqn (1)

$T(n) = T(n-2) + (n-1) + n \longrightarrow (3)$

Again $n = n-2$ in eqn (1)

$T(n-2) = T(n-2-1) + (n-2)$

$= T(n-3) + (n-2) \longrightarrow (4)$

Putting in eqn (3)

$T(n) = T(n-3) + (n-2) + (n-1) + n \longrightarrow (5)$

Now general eqn is,

$T(n) = T(1+2+3 \ldots n)$ for $T(n) = 1$

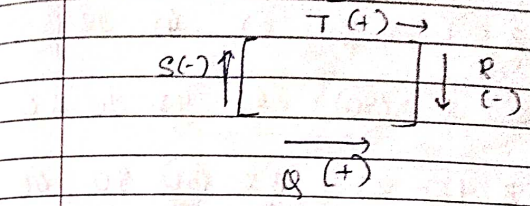
$= 1 + 2 + 3 \ldots n$

$T(n) = \frac{n(n+1)}{2}$

$= \frac{n^2+n}{2} = n^2 = \boxed{O(n^2)}$

Strassen's Matrix Multiplication

$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ $B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$



$P = (A_{11} + A_{22})(B_{11} + B_{22})$

$Q = B_{11}(A_{21} + A_{22})$

$R = A_{11}(B_{12} - B_{22})$

$S = A_{22}(B_{21} - B_{11})$

$T = B_{22}(A_{11} + A_{12})$

$U = (A_{21} - A_{11})(B_{11} + B_{12})$

$V = (B_{21} + B_{22})(A_{12} - A_{22})$

$C_{11} = P + S - T + V$

$C_{12} = R + T \longrightarrow$ Add

$C_{21} = Q + S$

$C_{22} = P + R - Q + V$

Quick Sort

checking ← [right → less
left → higher

pivot → (50) 3 1 60 65 45 90 13 67

13 3 1 60 65 65 90 (50) 67

13 3 1 (50) 65 45 90 60 67

13 3 1 45 65 (50) 90 60 67

13 3 1 45 | (50) | 65 90 60 67

pivot → (13) 3 1 45 | (65) 90 60 67

3 (13) 13 45 | 60 90 (65) 67

3 1 (13) 45 | 60 65 90 67

(3) 1 13 45

1 (3) 13 45

Merge Sort :-

(Refer from the classroom)

Ex : of Quick Sort :-

(9) 7 5 11 12 2 14 3 10 6

pivot changed 6 7 5 11 12 2 14 3 10 (9)

6 7 5 (9) 12 2 14 3 10 11

6 7 5 3 12 2 14 (9) 10 11

6 7 5 3 (9) 2 14 12 10 11

6 7 5 3 2 (9) 14 12 10 11

(6) 7 5 3 2 | (14) 12 10 11

2 7 5 3 (6) | 11 12 10 (14)

2 (6) 5 3 7 | (11) 12 10 14

2 3 5 (6) 7 | 10 12 (11) 14

2 3 5 6 7 | 10 (11) 12 14

2 3 5 6 7 9 10 11 12 14

Sorted form :-

2 3 5 6 7 9 10 11 12 14

(3) 7 12 10 1 4 15 3

pivot (7) 12 10 1 4 15 3

3 12 10 1 4 15 (7)

3 (7) 10 1 4 15 12

3 4 10 1 (7) 15 12

3 4 ~~10~~ (7) 1 ~~10~~ 10 15 12

3 4 1 (7) 10 15 12

(3) 4 1 (7) (10) 15 12

1 ~~4~~ (3) 10 (15) 12

1 3 4 7 10 12 15

1 3 4 7 10 12 15

Sorted array

1	3	4	7	10	12	15

☆ Greedy Algorithm

(1) Activity selection on Problem

Activity	A1	A2	A3	A4	A5	A6	A7	A8	A9
start	1	2	4	1	5	8	9	11	13
and finish	3	5	7	8	9	10	11	14	15

Step 1:-> Sort the activity according to the finish time.

Algorithm:->

```
Activity Selector (S, F)
  n = s.length
  A = {a1} // Activity selector set
  k = 1
  for m = 2 to n // n = 9
    if S[m] >= f[k]
      A = Au{am}
      k = m
  return A;
```

☆ Master Method

$$T(n) = a\,T\left(\frac{n}{b}\right) + \Theta\left(n^k \log^p n\right)$$

$a \geq 1, b > 1, k \geq 0, p$ = real no.

Case 1 :⇒ if $a > b^k$ then

$$T(n) = \Theta(n^{\log_b a})$$

Case 2 :– if $a = b^k$

i) if $P > -1$ then

$$T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$$

ii) if $P = -1$ then

$$T(n) = \Theta(n^{\log_b a} \log \log n)$$

iii) if $P < -1$ then

$$T(n) = \Theta(n^{\log_b a})$$

Case 3 :⇒ if $a < b^k$ then

i) if $P \geq 0$ then

$$T(n) = \Theta(n^k \log^p n)$$

ii) if $P < 0$ then

$$T(n) = O(n^k)$$

eg :⇒ $T(n) = 4T(n/2) + n$

⟶ $a = 4$, $b = 2$, $k = 1$, $P = 0$.

$a \geq b^k$

$4 \geq 2^1$

By ~~use~~ using 1st case of master Theorem

$$T(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(n^{\log_2 4})$$

$$= \Theta(n^{\log_2 (2)^2})$$

$$= \Theta(n^{2 \log_2 (2)})$$

$$= \Theta(n^{2 \log_2 (2)})$$

$\therefore \log_2 (2) = 1$

$$\boxed{T(n) = \Theta(n^2)}$$

(6) ⟶ $T(n) = 16^n T\left(\frac{n}{3}\right) + \frac{1}{n}$

$T(n) = 16^n T\left(\frac{n}{3}\right) + n^{-1}$

$a = 16^n$ $b = 3$ $k = -1$

(It can be solved by master theorem)

(5) ⟶ $T(n) = 3T(n/2) + n^2$.

$a = 3$ $b = 2$ $k = 2$ $P = 0$.

$a \geq 1$ $b \geq 1$ $k \geq 0$ $P =$ real no.

$a \quad b^k$.

$3 \quad 2^2$. $\therefore a < b^k$

$3 < 4$

By using Case 3 of master theorem

i) $P \geq 0$ then

$T(n) = \Theta(n^k \log^p n)$

$T(n) = \Theta(n^2 \log^0 n)$

$\boxed{T(n) = \Theta(n^2)}$

(4) $T(n) = 2T(n/2) + n \log n$

→ $a=2$, $b=2$, $k=1$, $p=1$

$a \geq 1$, $b \geq 1$, $k \geq 0$, p = real no.

$a = b^k$.

$2 = 2^1$

By using 2nd case of M.T.

(1) $p > -1$ then

$T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$

$= \Theta(n^{\log_2 2} \log^2 n)$

$\boxed{T(n) = \Theta(n \log^2 n)}$

(5) $T(n) = 2T(n/2) + n^2$

→ $a=2$, $b=2$, $k=2$, $p=0$.

$a \geq 1$, $b \geq 1$, $k \geq 0$, p = real no.

$a \; b^k$

$2 < 4$ Using 3rd case

(1) $p \geq 0$

$T(n) = \Theta(n^2 \log^0 n)$ $\boxed{T(n) = \Theta(n^2)}$

(6) $T(n) = 2T(n/2) + \frac{n}{\log n}$.

→ $a=2$ $b=2$ $k=1$ $p=-1$

$a \; b^k$.

$2 = 2^1$ $\boxed{2=2}$

By using 2nd case of M.T as $p = -1$

$T(n) = \Theta(n^{\log_b a} \log \cdot \log n)$

$T(n) = \Theta(n^{\log_2 2} \log \cdot \log n)$

$\boxed{T(n) = \Theta(n \log \cdot \log n)}$

(7) $T(n) = 3T(n/2) + \frac{n^2}{\log n}$

→ $a=3$ $b=2$ $p=-1$ $k=2$

$a \; b^k$

$3 < 2^2 = 4$

$\therefore p = -1$. By using 3rd case.

$T(n) = \Theta(n^k)$

$= \Theta(n^2)$

$\boxed{T(n) = \Theta(n^2)}$

(8) $T(n) = 2T(n/2) + n/\log^2 n$

→ $a=2$ $b=2$ $k=1$ $p=-2$

$\boxed{2=2}$ $\boxed{a = b^k}$ $p < -1$

$T(n) = \Theta(n^{\log_b a})$

$= \Theta(n^{\log_2 2}) = \underline{\Theta(n)}$

(3) $T(n) = 2T\left(\frac{n}{4}\right) + n^2 \log n$

→ $a = 2 \quad b = 4 \quad k = 2 \quad p = 1$

$T(n) = aT(n/b) + n^k \log^p n$

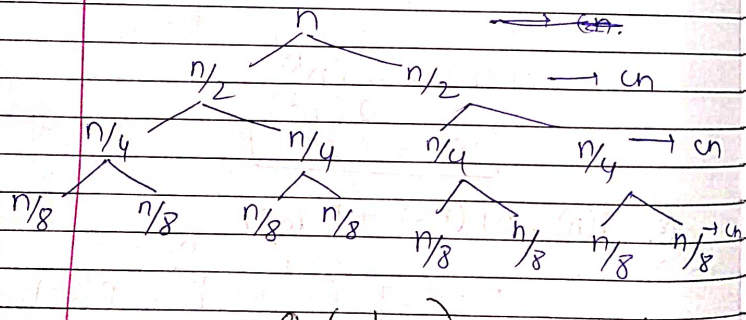
a	b^k	
2	4^2	$a < b^k$
2	16	

(i) $p \geq 0 \qquad T(n) = \Theta(n^k \log^p n)$

$T(n) = \Theta(n^2 \log n)$

✱ Recursive - Tree Method

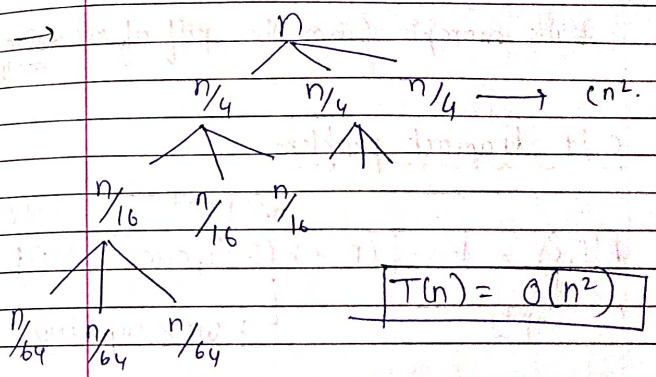
$T(n) = 2T(n/2) + cn$ $\quad \frac{n}{2} = \log n$



$= \Theta(n \log n)$ Height of tree = $\log n$

(2) $T(n) = 3T\left[\frac{n}{4}\right] + cn^2$

→



$T(n) = O(n^2)$

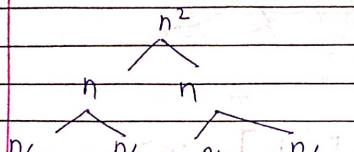
(4) $x^{\log_y n} = n^{\log_y x}$

$x^0 + x^1 + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$ for $x \neq 1$

$x^0 + x^1 + x^2 + \cdots + x^n = \frac{1}{1 - x}$ for $|x| < 1$

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + n^2 & n>1 \end{cases}$$

We assum that n is exact power of 2.



Write example from the pdf of recursion method.

☆ 0/1 knapsack problem

$$k(i,c) = \max\left[(p_i + k(i-1, c-w_i)), k(i-1, c)\right]$$

new profit, old profit

obj, capacity

↳ $w_i \le$ (capacity)

$= 0$, $i = 0$ or $c = 0$

$= k(i-1, c)$ ⟶ previous profit

↳ $w_i >$ capacity

i / capm	0	1	2	3	4	5
object 0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

for k(1,1).

$= \max\left[p_1 + k(1-1, 1-2), k(1-1, 1)\right]$

$= \max\left[p_1 + k(0,-1), k(0,1)\right]$

$= 0$

$k(1,2) = \max\left[(p_1 + k(1, 2-2), k(1-1, 2)\right]$

$= \max\left[(3) + k(0,0), k(0,2)\right]$

$= \max\left[3, 0\right]$

$= 3$.

Ex :-> $n = 3$. $m = 20$ (capacity)

object	1	2	3
Profit	25	24	15
weight	18	15	10

HW

(5) : P = { 1, 2, 5, 6 }

W = { 2, 3, 4, 5 }

M = 8 , n = 4

* Greedy Technique

* Job sequencing with Deadline

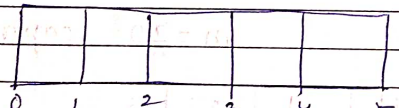
Job	J_1	J_2	J_3	J_4	J_5	J_6
Deadline	5	3	3	2	4	2
Profit	15	10	12	20	8	5
	2	4	3	1	5	6.

Step 1 :=> Job

Job	J_4	J_1	J_3	J_2	J_5	J_6
Dea	2	5	3	3	4	2
	20	15	12	10	8	5

(Arrange in descending order a/c to profit)

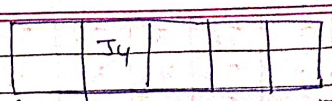
Step 2 :=>



max deadline.

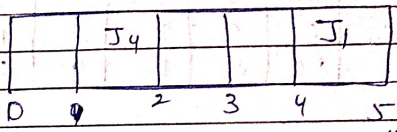
Step 3 :=> Select job into table a/c to max.m profit

J_4 profit = 20, D = 2



Profit = 20

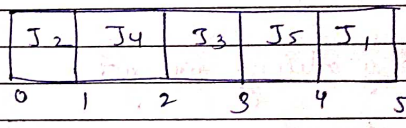
Step 4 :=> Consider J_1 . P = 15 D = 5.



Profit = 20 + 15

By writing all the table step by step

Step 5 :=> Consider J_3 P = 12 D = 3.



Profit = 20 + 15 + 12 + 10 + 8

Profit = 65

Page No.
Date : / /

Q.

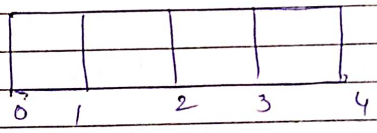
Job	J_1	J_2	J_3	J_4	J_5	J_6	J_7
Deadline	1	3	4	3	2	1	2
Profit	3	5	20	18	9	6	30

→ Arrange in descending order of to profit

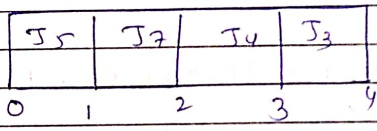
1) :->

Job	J_7	J_3	J_4	J_5	J_6	J_2	J_1
Deadline	2	4	3	2	1	3	1
Profit	30	20	18	9	6	5	3

2) :-> Max^m Deadline.



By soluing we get.



Profit = 9 + 18 + 20 + 30
= 77

Page No.
Date : / /

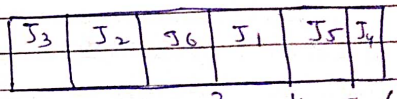
Q.

J_1	J_2	J_3	J_4	J_5	J_6	J_7
4	2	2	6	5	3	1
60	40	20	70	50	30	10

→

1) :->

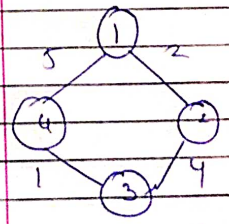
J_4	J_1	J_5	J_2	J_6	J_3	J_7
6	4	5	2	3	2	1
70	60	50	40	30	20	10



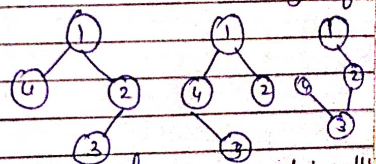
Profit = 70 + 60 + 50 + 40 + 30 + 20
= 270.

✱ MST (Minimum cost spanning Tree)

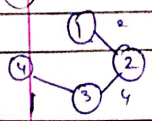
✱ Prism & Kruskal



spanning tree = n-1 vertex and non-cyclic form



Prism = (7)



Kruskal :- arrange i/c to small edge

